

ELO329 - Diseño y Programación Orientados a Objetos

`const` , referencias y `assert` en C++

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

En la clase anterior vimos cómo declarar clases, separar `.h` y `.cpp`, y construir objetos.

Ahora veremos cómo hacer que esas clases sean más seguras y expresivas:

- `const` permite declarar qué no debe cambiar.
- Las referencias permiten trabajar con objetos existentes sin copiarlos.
- `assert` permite verificar supuestos internos durante depuración.

El foco de esta clase es escribir código donde las intenciones queden explícitas para el compilador y para quien lee.

Contenidos

1. Calificador `const`
2. `const` en variables y parámetros
3. `const` en punteros
4. Métodos `const`
5. Referencias
6. Paso por referencia constante
7. Alcance de variables
8. Verificación con `assert`

La idea de const

`const` expresa una promesa: este valor no debe modificarse después de inicializarse.

```
const int n = 25;  
n = 36; // error
```

La promesa la revisa el compilador.

Esto hace que el error aparezca antes de ejecutar el programa.

Por qué usar const

`const` sirve para comunicar intención y evitar cambios accidentales.

Casos comunes:

- Constantes con tipo y alcance definido.
- Parámetros que no deben modificarse.
- Métodos que solo observan el objeto.
- Punteros que no deben moverse o datos apuntados que no deben cambiar.

Primero veremos el caso simple: una variable constante.

Variables constantes

Un objeto constante debe recibir un valor al definirse.

```
const int n = 25;  
int m = n;
```

```
m = 36; // ok  
n = 36; // error
```

`m` recibe una copia del valor de `n`.

Modificar `m` no modifica `n`.

Inicialización obligatoria

Una constante no puede quedar sin valor inicial.

```
const double z; // error
```

El motivo es directo: si no se puede modificar después, debe quedar lista desde su definición.

En C++, inicializar bien los objetos es parte central del diseño.

const y #define

Para constantes simples, `const` suele ser preferible a `#define`.

```
const int maxStudents = 80;
```

Ventajas:

- Tiene tipo.
- Respeta alcance.
- El compilador puede revisar mejor su uso.
- Es más legible en código C++.

El siguiente problema: punteros

Con punteros aparece una pregunta adicional.

```
¿Quiero proteger el puntero?  
¿Quiero proteger el dato apuntado?
```

No es lo mismo impedir que el puntero cambie de dirección que impedir que cambie el valor al que apunta.

Dos formas distintas

```
int value = 4;  
int other = 7;  
  
int * const a = &value; // puntero constante a int  
const int *b = &value;  // puntero a int constante
```

Lectura:

- `int * const a` protege el puntero.
- `const int *b` protege el contenido apuntado.

La posición de `const` cambia qué queda protegido.

Puntero a dato constante

```
void MySub(const int *a) {  
    *a = 50; // error  
    a++;    // ok  
}
```

En este caso:

- No puedo modificar el contenido usando `*a`.
- Sí puedo mover el puntero local `a`.

El puntero fue pasado por valor, por lo que moverlo dentro de la función no cambia el puntero original.

Puntero constante

```
void MySub(int * const a) {  
    *a = 50; // ok  
    a++;    // error  
}
```

En este caso:

- Sí puedo modificar el dato apuntado.
- No puedo hacer que `a` apunte a otra dirección.

La promesa ahora protege la variable puntero, no el contenido.

Resumen de punteros y const

Declaración	Protege	Permite cambiar <code>*a</code>	Permite cambiar <code>a</code>
<code>const int *a</code>	Dato apuntado	No	Sí
<code>int * const a</code>	Puntero	Sí	No
<code>const int * const a</code>	Ambos	No	No

La misma idea de promesa se puede aplicar ahora a métodos de una clase.

const en métodos

Un método `const` promete no modificar el estado observable del objeto.

```
class Automobile {  
public:  
    void display() const;  
    int get_NumDoors() const;  
};
```

Esto es especialmente natural para accesoros.

Un accesor debe observar el objeto, no cambiarlo.

Efecto en cadena

Dentro de un método `const`, solo podemos llamar métodos que también respeten esa promesa.

```
void ShowAuto(const Automobile &aCar) {  
    aCar.display();  
}
```

Si `display` no está declarado como `const`, el compilador no puede garantizar que `aCar` no cambie.

Por eso se produce un error.

const como parte de la interfaz

`const` no es solo un detalle de implementación.

Forma parte del contrato público de la clase.

```
void display() const;
```

Este contrato dice:

- El método puede llamarse sobre objetos constantes.
- El método no modificará el objeto.
- Otros métodos `const` pueden invocarlo con seguridad.

Ejemplo con CRectangle

El ejemplo del directorio usa una clase simple.

```
class CRectangle {  
    int width, height;  
  
public:  
    CRectangle();  
    CRectangle(int, int);  
    int area(void) { return width * height; }  
};
```

`area` observa el rectángulo, por lo que podría declararse como método `const`.

Mejora con método const

```
class CRectangle {  
    int width, height;  
  
public:  
    CRectangle();  
    CRectangle(int, int);  
    int area() const { return width * height; }  
};
```

Con esta firma, `area` puede usarse sobre rectángulos constantes o recibidos por referencia constante.

Referencias

Una referencia es un alias para un objeto existente.

```
int N = 25;  
int &rN = N;  
  
rN = 36;  
std::cout << N << std::endl; // imprime 36
```

rN no es una copia de N.

rN es otro nombre para el mismo objeto.

Referencias frente a punteros

Aspecto	Puntero	Referencia
Sintaxis de acceso	*p o p->x	Igual que el objeto
Puede reasignarse	Sí	No
Puede ser nulo	Sí	No en uso normal
Debe inicializarse	No siempre	Sí

Las referencias permiten semántica similar a un puntero, pero con una sintaxis más cercana al objeto directo.

Alcance de variables

Una variable local existe solo dentro del bloque donde fue declarada.

```
int a = 10;  
  
{  
    int i = 20;  
    a += i;  
}  
  
// aquí i ya no existe
```

Un bloque queda delimitado por `{` y `}`.

Por qué importa el alcance

El alcance define cuándo nace y cuándo deja de existir una variable local.

Esto conecta con referencias:

- Una referencia debe apuntar a un objeto existente.
- No debemos devolver referencias a variables locales.
- Un alias no prolonga la vida del objeto original.

Ahora podemos combinar referencias y `const` para pasar objetos de forma eficiente y segura.

Paso por referencia

Una referencia permite que una función trabaje con el objeto original.

```
void scale(CVector &v) {  
    v.x *= 2;  
    v.y *= 2;  
}
```

Aquí `v` es el objeto original.

Por eso los cambios quedan visibles fuera de la función.

Referencia constante

Si queremos evitar copias, pero no queremos modificar el objeto, usamos referencia constante.

```
void ShowAuto(const Automobile &aCar) {  
    std::cout << "Example of an automobile: ";  
    aCar.display();  
}
```

Lectura:

- `&` evita copiar el objeto.
- `const` impide modificarlo dentro de la función.

Patrón común en C++

Para objetos grandes, esta forma es muy usada.

```
void process(const MiClase &obj);
```

Ventajas:

- Evita el costo de copiar.
- Protege el objeto recibido.
- Documenta que la función solo necesita observar.

Este patrón aparece con frecuencia en clases, contenedores y operadores.

Ejemplo con CVector

El ejemplo `cvector` usa referencia constante en la sobrecarga del operador `+`.

```
class CVector {  
public:  
    int x, y;  
    CVector();  
    CVector(int, int);  
    CVector operator+(const CVector &) const;  
};
```

Hay dos promesas en la misma línea.

Lectura de operator+

```
CVector CVector::operator+(const CVector &param) const {  
    CVector temp;  
    temp.x = x + param.x;  
    temp.y = y + param.y;  
    return temp;  
}
```

Lectura:

- `param` se recibe por referencia constante.
- El método completo termina en `const`.
- La suma no modifica ni `param` ni el objeto izquierdo.
- El resultado se entrega como un nuevo `CVector`.

Uso de CVector

```
CVector a(3, 5);  
CVector b(1, 2);  
CVector c;  
  
c = a + b;
```

La expresión `a + b` equivale conceptualmente a:

```
c = a.operator+(b);
```

`a` y `b` no deben cambiar por hacer una suma.

De promesas a verificaciones

Hasta ahora, `const` permite que el compilador revise promesas de no modificación.

Pero hay condiciones que dependen de valores en tiempo de ejecución.

Ejemplos:

- Un índice debe estar dentro de rango.
- Un número de años no debe ser negativo.
- Una tasa no debe ser negativa.

Para esos casos aparece `assert` .

assert

`assert` verifica una condición que debería ser verdadera si el programa está correcto.

```
#include <cassert>

assert(nyear >= 0);
assert(p >= 0);
```

Si la condición es falsa, el programa se detiene con un mensaje de diagnóstico.

Ejemplo de precondiciones

```
#include <cassert>
#include <cmath>

double future_value(double initial_balance,
                    double p,
                    int nyear) {
    assert(nyear >= 0);
    assert(p >= 0);

    double b = initial_balance
        * std::pow(1 + p / (12 * 100), 12 * nyear);
    return b;
}
```

Las llamadas a `assert` documentan supuestos internos del cálculo.

Qué ocurre si falla assert

Si la expresión es falsa, el programa se detiene inmediatamente.

Un mensaje típico indica:

```
Assertion failure in file mysub.cpp,  
line 201: nyear >= 0
```

`assert` sirve para depuración.

No es un mecanismo de recuperación de errores.

assert y NDEBUG

El efecto de `assert` puede eliminarse al compilar definiendo `NDEBUG`.

```
#define NDEBUG  
#include <cassert>
```

También puede definirse desde el compilador.

```
g++ -DNDEBUG programa.cpp
```

En ese caso, las verificaciones con `assert` no generan código efectivo.

Cuándo usar assert

Usar `assert` para supuestos que indican errores del programador:

- Precondiciones internas.
- Invariantes de una clase.
- Estados que deberían ser imposibles.

No usarlo para errores esperables del entorno:

- Entrada inválida de usuario.
- Archivo inexistente.
- Falla de red.

Esos casos requieren manejo de errores.

Conexión entre los temas

Los tres conceptos ayudan a escribir código más claro.

Concepto	Qué expresa	Quién lo revisa
<code>const</code>	Esto no debe cambiar	Compilador
Referencia	Esto es un alias, no una copia	Compilador
<code>assert</code>	Esto debe ser verdadero al ejecutar	Programa en depuración

La idea común es hacer explícitos los supuestos del código.

Ideas clave

- `const` declara promesas de no modificación.
- En punteros, `const` puede proteger el puntero, el dato apuntado o ambos.
- Un método `const` promete no cambiar el estado del objeto.
- Una referencia es un alias para un objeto existente.
- `const T &` evita copias y protege el objeto recibido.
- El alcance determina la vida de variables locales.
- `assert` verifica precondiciones durante depuración.
- `assert` no reemplaza el manejo normal de errores.